

SoMeSci—A 5 Star Open Data Gold Standard Knowledge Graph of Software Mentions in Scientific Articles

David Schindler

david.schindler@uni-rostock.de

Institute of Communications Engineering, University of
Rostock
Rostock, Germany

Stefan Dietze

stefan.dietze@gesis.org

GESIS - Leibniz Institute for the Social Sciences
Cologne, Germany
Heinrich-Heine-University
Düsseldorf, Germany

Felix Bensmann

felix.bensmann@gesis.org

GESIS - Leibniz Institute for the Social Sciences
Cologne, Germany

Frank Krüger

frank.krueger@uni-rostock.de

Institute of Communications Engineering, University of
Rostock
Rostock, Germany

ABSTRACT

Knowledge about software used in scientific investigations is important for several reasons, for instance, to enable an understanding of provenance and methods involved in data handling. However, software is usually not formally cited, but rather mentioned informally within the scholarly description of the investigation, raising the need for automatic information extraction and disambiguation. Given the lack of reliable ground truth data, we present SoMeSci—Software Mentions in Science—a gold standard knowledge graph of software mentions in scientific articles. It contains high quality annotations (IRR: $\kappa=.82$) of 3756 software mentions in 1367 PubMed Central articles. Besides the plain mention of the software, we also provide relation labels for additional information, such as the version, the developer, a URL or citations. Moreover, we distinguish between different types, such as application, plugin or programming environment, as well as different types of mentions, such as usage or creation. To the best of our knowledge, SoMeSci is the most comprehensive corpus about software mentions in scientific articles, providing training samples for Named Entity Recognition, Relation Extraction, Entity Disambiguation, and Entity Linking. Finally, we sketch potential use cases and provide baseline results.

CCS CONCEPTS

• **Computing methodologies** → **Information extraction**; • **Information systems** → **Resource Description Framework (RDF)**.

KEYWORDS

Knowledge Graph, Software Mention, Named Entity Recognition, Relation Extraction, Entity Disambiguation, Entity Linking



This work is licensed under a Creative Commons Attribution International 4.0 License.

CIKM '21, November 1–5, 2021, Virtual Event, QLD, Australia

© 2021 Copyright held by the owner/author(s).

ACM ISBN 978-1-4503-8446-9/21/11.

<https://doi.org/10.1145/3459637.3482017>

ACM Reference Format:

David Schindler, Felix Bensmann, Stefan Dietze, and Frank Krüger. 2021. SoMeSci—A 5 Star Open Data Gold Standard Knowledge Graph of Software Mentions in Scientific Articles. In *Proceedings of the 30th ACM International Conference on Information and Knowledge Management (CIKM '21)*, November 1–5, 2021, Virtual Event, QLD, Australia. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3459637.3482017>

1 INTRODUCTION

Software plays a significant role in today's research and is thus often mentioned in scholarly articles. It is mentioned for different purposes, including its actual usage or the introduction of a novel software, see Fig. 1. Knowledge about software used in scientific investigations is important for several reasons, e.g. to provide an understanding of provenance of the research results or to give credit to its developers. Moreover, as software is a key element of data-driven science, knowledge about the particular version, i.e. software development state, is a necessary prerequisite for the reproducibility of the scientific results as even minor changes to the software might impact them significantly. Moreover, information about newly created software is used in publicly funded projects, such as the OpenAIRE Knowledge Graph [16], aiming at linking Open Access articles to their respective data and software. Enabling the automatic identification of software creation statements would allow to automatically include such information.

Despite the existence of software citation principles [12, 26], software mentions in scientific articles are usually informal and often incomplete—information about the developer or the version are often missing entirely, see Fig. 1. Spelling variations and mistakes for software names, even common ones [25], increase the complexity of automatic detection and disambiguation. Training and evaluation of information extraction approaches requires reliable ground truth data of sufficient size, raising the need for manually annotated gold standard corpora of software mentions. Available corpora [3, 4, 25] do not cover all available information, omitting additional information or disambiguation for different spelling variations of the same software. Moreover, no available corpus distinguishes between the purpose of the mention such as creation or usage (see Fig. 1).

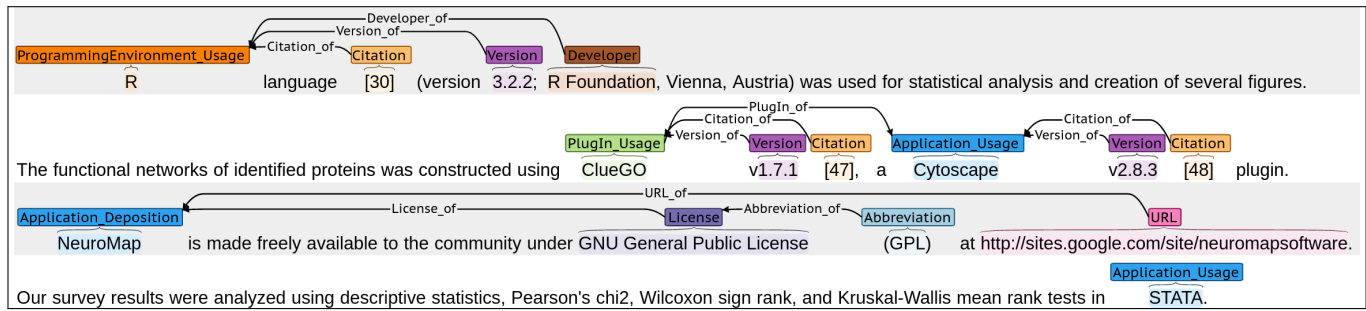


Figure 1: Example of software mention statements of different completeness and complexity annotated according to the conceptual model described in Section 3.1.

Here, we present SoMeSci, a gold standard knowledge graph of software mentions in scientific articles. We distinguish between different types of software and mention, provide additional information and disambiguate spelling variations, providing—to the best of our knowledge—the most comprehensive gold standard corpus of software mentions in scientific articles. SoMeSci is created by manually annotating 3756 software mentions with additional information, resulting in 7237 labelled entities in 47,524 sentences from 1367 PubMed Central articles. Data is lifted into a knowledge graph by using established vocabularies such as NLP Interchange Format (NIF) [8] and schema.org [22], disambiguated and linked to external resources, and shared as a publicly available 5-star Linked Open Data resource as defined by Berners-Lee [2] that can be explored interactively. An excerpt of the KG based on the first sentence in Fig. 1 is illustrated in Fig. 2.

As shown in Section 6, whereas entity recognition, relation extraction or entity disambiguation with focus on software mentions in scholarly articles require high quality data for training and testing, SoMeSci provides the most comprehensive ground truth dataset so far, able to advance progress in the aforementioned areas. We also demonstrate that methods trained with our corpus and their application to unseen corpora of scholarly articles are able to obtain significant insights into software citation practices.

In the following sections we give an overview of existing corpora for software mentions and introduce a conceptual model for software descriptions in scientific articles and our particular implementation. Then, we describe our annotation process and the SoMeSci corpus that resulted from it. We present reproducible use-cases, discuss the impact, establish baseline results and assess its generalization. Finally, we discuss limitations and conclude.

2 RELATED WORK

Identification of software mentions in scholarly articles gained momentum over the past years. Until recently, only few data was available [13]. The results of manual content analyses with respect to software in scientific articles had been discussed [1, 9, 18], and, moreover, only one labeled resource of limited size was available: BioNerDs [4] consists of 85 annotated articles and covers mentions of software and databases in the life sciences with a particular focus on bioinformatics. Only recently, additional resources became available: SoSciSoCi [25] provides 1380 software mentions from 480 methods sections and additional 1005 from individual sentences of

Table 1: Comparison of the different available corpora for software mentions in scholarly articles. Domain: BI=Bioinformatics, L=Life Sciences, E=Economics, S=Social Science; IRR: F=FScore, O=Overlap agreement, κ =Cohen’s κ ; $\dagger$ =not all negative sentences were published; * =The original paper reported social science as the domain, but adds a remark about possible deviation to the limitations

	BioNerDs	SoftCite	SoSciSoCi	SoMeSci
# pos. Sentences	1,845	3,305	1,809	2,728
# neg. Sentences	19,120	$\dagger$ 13,069	28,045	44,796
# Software	2,625	4,093	2,385	3,756
# Other	0	2544	0	3,481
Software Type	☐	☐	☐	☒
Mention Type	☐	☐	☐	☒
Developer	☐	☒	☐	☒
Version	☐	☒	☐	☒
URL	☐	☒	☐	☒
Citation	☐	☐	☐	☒
Abbreviation	☐	☐	☐	☒
Extension	☐	☐	☐	☒
License	☐	☐	☐	☒
Disambiguation	☐	☐	☐	☒
Entity Linking	☐	☐	☐	☒
Source Domain	BI	L,E	S*	L
Source Year	2002–10	2000–10	2007–19	2007–20
IRR	$F=.80$	$O=.76$	$\kappa=.82$	$\kappa=.82$, $F=.93$

677 of PLoS articles. Most recently, Softcite [3] was presented with software mentions from 4971 articles split into 2521 biomedicine and 2450 economics articles. A detailed comparison of the available corpora is provided in Table 1.

While those corpora have improved the available data for software mentions in scientific articles, there are still aspects not yet covered. BioNerDs and SoSciSoCi do not take into account information associated with software mentions while Softcite considers *publisher*, *version* and *url*. None of the sets distinguish between different types of software and no dataset considers the type of

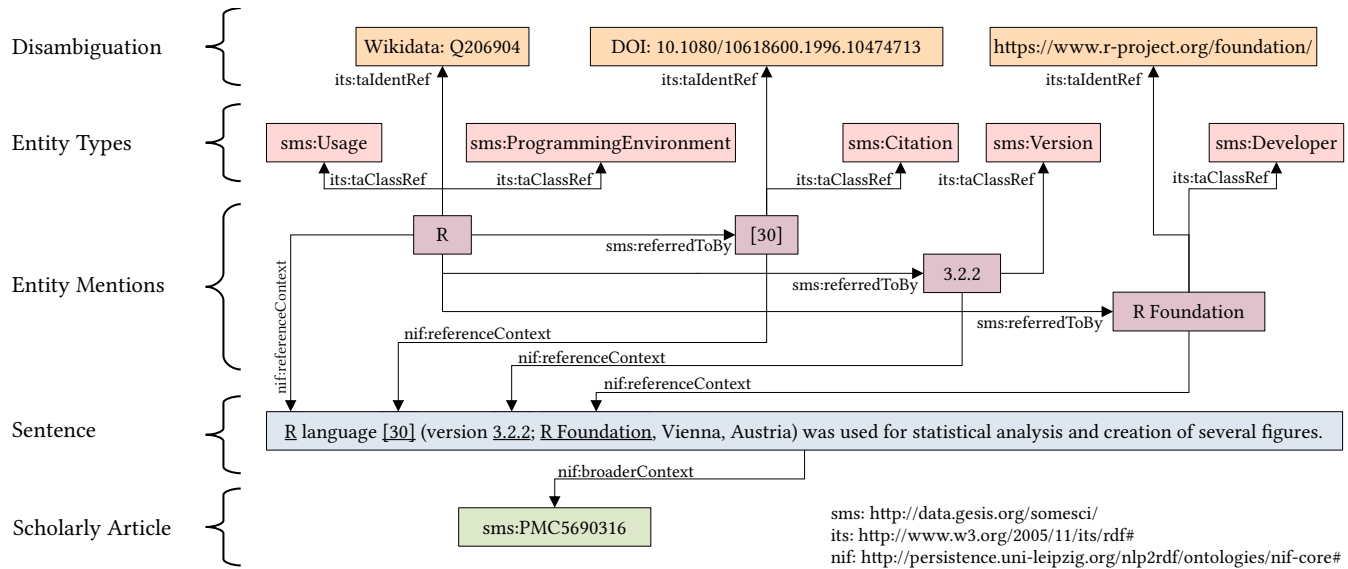


Figure 2: Excerpt of the SoMeSci knowledge graph illustrates the textual references of software mentions and their version, developer and citation. The different levels of representation separate the main concerns of interest, natural language sentences, mentions of entities, their types and disambiguation to knowledge entities. For clarity, some information is omitted.

mention. Further, no dataset provides any means to disambiguate the software names other than the plain name, which was shown to be insufficient for software disambiguation [25], see also Section 6.1.

Considering recent paradigm shifts towards data driven analyses across all disciplines making software a first class citizens of science and the resulting changes in awareness about software citations, the publication date of the analysed articles is important to allow conclusions. While BioNerDs and Softcite articles were both published before 2011, SoSciSoCi contains articles published between 2007 and 2019.

With respect to automatic identification of software mention a similar shift towards supervised learning could be observed [13]. While Pan et al. [19] used iterative bootstrapping due to a lack of available data, Duck et al. [4] employed manually constructed rules, which were later improved by additional machine learning classifiers [5]. More recently, machine learning based on Conditional Random Fields (CRFs) [3] and Bi-LSTM-CRFs [25] has been used.

Knowledge Graphs have been used for the representation of scholarly knowledge to handle the huge amount of information around scientific articles and hidden in the textual descriptions within scientific publications [11]. The Microsoft Academic Knowledge Graph (MAKG) [6] covers information about scientific publications and its related meta-data such as authors, institutions, and more. The Open Research Knowledge Graph [11] represents information about scientific investigations to enable the selection and comparison of those. OpenAIRE [16] strives to link scholarly articles with their published data and software. The automatic construction of such knowledge graphs, however, relies on the existence of automatic information extraction methods trained and evaluated with reliable ground truth data.

In summary, a high quality annotated corpus of software mentions in recently published, scholarly articles providing labels for

(1) types of software, (2) types of mention, (3) additional information, as well as (4) disambiguation, is missing but necessary to apply methods of automatic information extraction for automatic, large scale knowledge graph construction.

3 DATA MODEL

Software is mentioned within scholarly articles with different levels of detail [9], e.g. including version or developer, and for different purposes, e.g. usage of an existing application or description of a novel software as illustrated in Fig. 1. Different types of software can be distinguished, ranging from end-user applications to plugins or programming environments [14] (see Fig. 1), each of them contributing to the investigation in a different way. The data model was designed to cover these three dimensions by distinguishing between **Type of Software**, **Type of Mention**, and **Additional Information**. In the following, both, the conceptual model and the particular implementation are described.

3.1 Conceptual Model

The model captures the intricacies of in-text software mentions in scholarly articles. Definitions for all considered software entities, additional information, and their relationships are provided.

3.1.1 Type of Software. Li et al. [14] state that beside end-user applications, software packages are commonly used within scientific investigations and propose the categories of *software* and *package*. Further extending these categories, we distinguish the following software types:

Application is a standalone program, designed for end-users. Using applications usually results in data or project files associated with it, e.g. Excel sheets. This definition includes web-based applications such as web-services.

Plugin is an extension to a software that cannot be used individually. Often, the original application can be concluded from the plugin, e.g. ggplot2 is a well known R package.

Operating System (OS) is a special type of software that is used to manage the hardware of a computer and all other software processes running on it.

Programming Environment (PE) is an integrated environment that is built around programming languages and is used to design programs or scripts. This implicitly includes compilers or interpreters.

Distinguishing these software types allows deeper analysis of software usage in scientific investigations, a plugin requires, for instance, the corresponding application. Software mentions are, therefore, related to one another in two cases: (1) If both, the plugin and the respective application, are mentioned they are considered to be in a *plugin of* relation, and (2) if both mentions refer to the same software entity but one provides more additional information it is considered to be a *specification of* the other mention.

3.1.2 Type of Mention. Multiple reasons for software mentions within scholarly articles exist. Howison and Bullard [9] distinguish between *mention* and actual *usage* of a software. Based on the further extended categorization scheme of Schindler et al. [24], we distinguish the following hierarchy of mention types:

Allusion describes each mention of a software name. It does not require an indication of actual usage, e.g. stating a fact about the software, or comparing multiple suited software for a problem. Software allusions are comparable with scholarly citations used to refer to related work.

Usage (sub-type of Allusion) indicates that a software was used, and, therefore, made an actual contribution to the study. Software usage statements allow conclusions regarding provenance.

Creation (sub-type of Allusion) indicates the introduction of a new software. Software creation statements can be used to conclude the authors of the software and thus to provide scholarly credit to them.

Deposition (sub-type of Creation) indicates the publication of a new software. It extends a software creation statement by stating details about the publication by either providing a license or URL. In the scope of depositions, co-references are used to annotate indirect statements about software when providing details on availability and licensing.

The mention type can be used to draw conclusions about the role and availability of the software.

Note that an actual software statement within scholarly articles is a combination between a *Type of Software* and *Type of Mention*.

3.1.3 Additional Information. According to software citation principles [12, 26], software mentions in scholarly articles should be accompanied by additional information that allows the unique identification of the actually used software. However, articles often lack this information [9]. Based on a prior analysis of software mention statements [25], we distinguish between the following additional information about software provided in scholarly articles.

Developer is the person or organization that developed and published the software. Developers can change over time.

Version indicates a defined state in the software life-cycle. The version is usually provided as a number based identifier. As functionality and implementation changes over time, the version is one key element in reproducibility.

URL provides a location where additional information can be obtained.

Citation is formal bibliographic citation connected to the software mention.

Extension indicates different function ranges of a software, for instance, distinguishing a professional version from the basic version of a software.

Release is a published version of a software that, in difference to versions, is released using a date based identifier.

License declares a software's permissions and terms of usage.

Abbreviation is a shortened form of the software name, often an acronym.

Alternative name is the opposite of abbreviation and describes a longer name for a given software. Often a software is referred to by its acronym but a more complex, alternative name of the software is provided for completeness.

The additional information described above is used to specify other entities and is, therefore, always related to another entity. Most commonly, they are related to software mentions, but licenses can also be accompanied by a version, a URL or an abbreviation as well as developers who can be specified by a URL or an abbreviation.

3.1.4 Entity Identities. Each textual mention describes a particular software, see for instance Fig. 2, providing the actual entity identity. Beside software, such identities do also exist for developers, licenses and citations.

3.2 RDF/S Vocabulary

Objective of the data model is to represent the textual information, including their relations provided from the scholarly articles. This does not necessarily reflect the truth in case incorrect information is provided by the software mention. Moreover, as the corpus shall provide a means for training of information extraction and disambiguation methods, modeling was done based on textual entities. To this end, the data model was based on NIF [8], rather than vocabulary for the description of software such as the Software Ontology [15], OKG-Soft [7], and SoftwareKG [25]. The data model is illustrated in Fig. 3.

In detail, each sentence is represented by type `nif: Sentence`, `nif: Context` and `nif: OffsetBasedString`, providing the properties to embed the entity into a `nif: broaderContext` such as the source article. The particular position of the respective sentence within the `nif: broaderContext` is given by `nif: beginIndex` and `nif: endIndex`, to allow the reconstruction of the source text (see Section 4.1) to facilitate using the resource for other NLP-based analyses. The actual text of the sentence is provided by `nif: isString`.

Labelled entities are represented by objects of types `nif: Phrase` and `nif: OffsetBasedString`, and provide properties for the location of the actual mention within the sentence. As can be seen from Fig. 3, `sms: referredToBy` (`rdfs: subPropertyOf` of `nif: inter`) reflects the associations between the respective software (or license or developer) mention and the provided additional information.

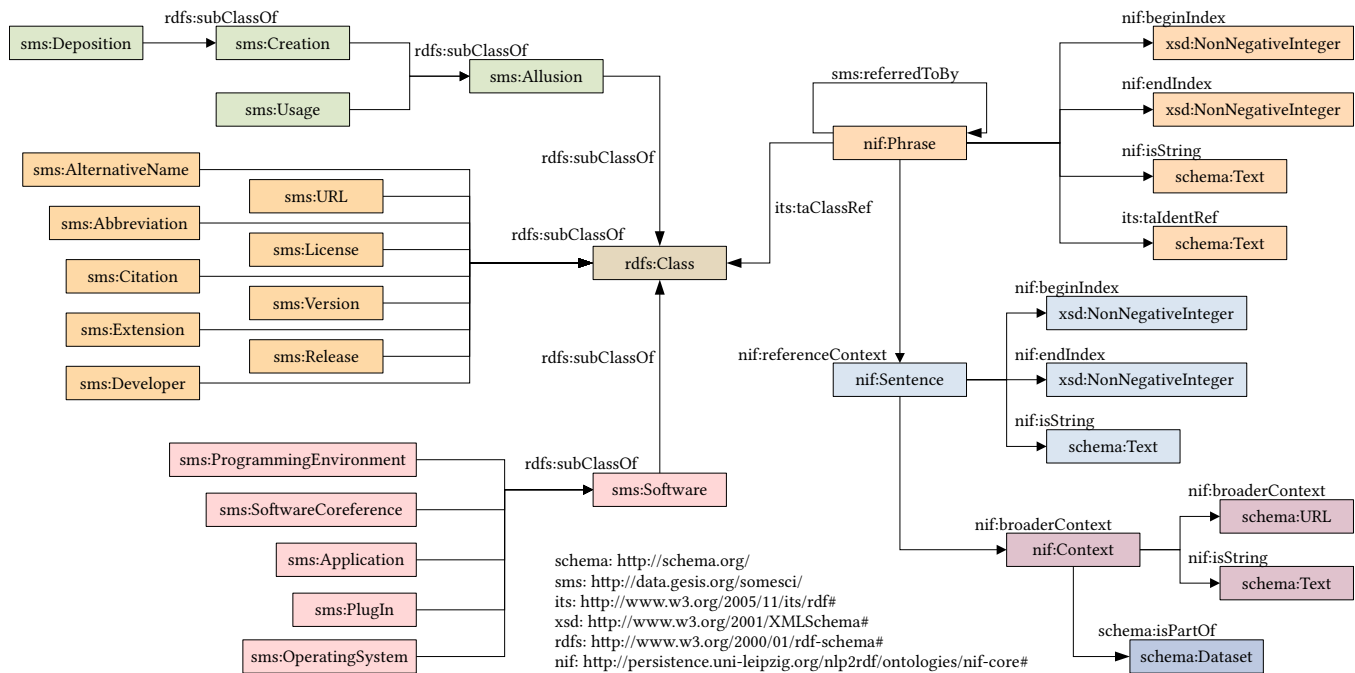


Figure 3: Overview of the data model. Note that properties specialised via `rdfs:subPropertyOf` of `sms:referredToBy` are used to represent the actual type of relations between entity mentions (`nif:Phrase`).

Class labels, e.g. Application or Usage, were provided via `its:taClassRef`, where each entity type in Section 3.1 is represented by an individual `rdfs:Class`. Software type labels represent a combination of software type and mention type, and are thus linked to both. Furthermore, `nif:anchorOf` provides the actual entity mention and `nif:referenceContext` links to the sentence with the current mention. To facilitate entity disambiguation and linking, each entity of classes Software, Developer, License, and Citation provides a link to the corresponding entity with `nif:taIdentRef`, as described in Section 4. Fig. 2 illustrates an example.

The labelled text is lifted to RDF by using `rdflib` v5.0.0 and serialized into JSON-LD with `rdflib-jsonld` v0.5.0 by using version 3.7.3 of the Python programming environment. The descriptive statistics of the dataset is given below.

4 LABELING SOFTWARE IN SCIENTIFIC ARTICLES

4.1 Article Selection

A broad range of scientific publications was selected to establish a corpus of articles. All selected articles are available from PubMed Central (PMC) Open Access (OA) subset, the largest collection of OA articles. PMC articles are available in an XML format and were obtained via bulk download on January 22, 2021. The main focus of journals indexed in PMC are life sciences, but some, such as PLoS, also publish interdisciplinary articles of other domains.

Part of the corpus was created by re-labelling the SoSciSoCi corpus [25] that only covers a small aspect of the conceptual model (see Section 2 and Section 3.1). Performing a re-annotation of an

existing corpus has the benefit of enhancing quality by reducing the number of missed annotations and enforcing consistency between all annotations because articles are read and annotated twice. The respective provenance is documented via `prov:wasDerivedFrom` in the datamodel. Overall, the corpus contains 1367 articles split into the following four groups:

PLoS methods 480 methods sections from PLoS journal articles originating from SoSciSoCi. Note that while the original study targeted at the social sciences, Fig. 4 illustrates a more diverse distribution of the research domains.

PLoS sentences Individual sentences from 677 PLoS articles selected based on the appearance of common software names originating from SoSciSoCi.

PubMed fulltext 100 randomly selected full text articles from PMC OA to allow generalization assessment from methods sections. 100 full text articles was considered sufficient as the annotation effort increases with the length of articles.

Creation sentences Individual sentences from 110 articles that contain software creation statements, selected from PMC OA (n=50) and PLoS (n=60). This set was specifically selected to represent the rare but important group of software creation and depositions which are necessary to gain an overall understanding of the software infrastructure in science (see Section 3.1). Articles were selected by analyzing titles and abstracts. Matched articles were manually verified and added to the set. Individual sentences were then selected by manually review.

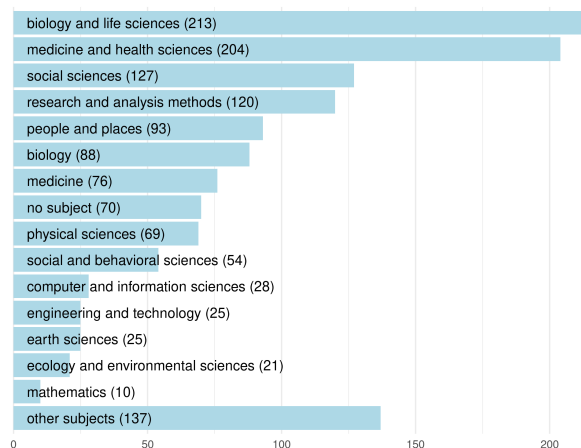


Figure 4: Research subjects of articles in PLoS methods.

4.2 Annotation Process

The conceptual model (see Section 3.1) was implemented in the web-based annotation software BRAT v1.3 [27] that was used to perform all textual annotations, see Fig. 1. The textual annotation process was split into two steps to reduce complexity and minimize the risk for errors during annotation: (1) all articles were initially annotated for software type, mention type and relations between two software mentions, and (2) additional information was added for all software annotations resulting from the first step. For annotation, the articles were split between two of the authors, both having a background in computer science and experience in working with software mentions. Additionally, inter-rater reliability (IRR) was assessed by performing overlapping annotations (see Section 4.3), and resulting differences were discussed and afterwards merged.

After annotation, all *software*, *developers*, *citations* and *licenses* entities were manually disambiguated to unique identifiers on mention basis, partitioned to 4 annotators. As sources for identifiers we considered WikiData, package repositories such as CRAN or PyPi, Github, and websites in the stated priority. As the initial IRR was found to be insufficient (see Section 4.3), disambiguation was repeated by 2 annotators, each working on the entire set of mention entities. Results of both were merged ensuring that all links to repositories were adjusted to the suggested canonical form.

We estimate the overall effort with about 480 hours. The annotation of PLoS methods sections took up the largest part with 320 hours. We estimate an average duration of 20+5 minutes for the two annotation runs for each article followed by 15 minutes of disambiguating 3 software mentions on average, incl. their developers and citations. Annotation of PMC fulltexts took 112 hours with an estimated 45 minutes for both annotation passes followed by 22 minutes for disambiguation. For annotating individual sentences we estimate an average of 3 minutes per article and for creation sentences 10 minutes totalling in 34 and 18 hours, respectively.

4.3 Inter-rater Reliability

IRR was determined on 10% overlap of articles from PLoS methods, PLoS sentences and PubMed fulltexts as described in Section 4.1.

Table 2: IRR of software and mention type. Note that n can differ between both annotations and is provided to give an estimate of the number of annotations. Only matching the most specific sub-type is considered correct for mention type.

Software ($\kappa=0.82$)			Mention ($\kappa=0.79$)		
Type	F1	n	Type	F1	n
Application	0.94	267	Usage	0.95	315
Plugin	0.72	26	Allusion	0.84	10
OS	1.00	8	Creation	1.00	4
PE	1.00	28	Deposition	–	0
Overall	0.93	329	Overall	0.95	329

Table 3: IRR for additional information about software. Note that plugin and specification are relations between software entities.

Recognition ($\kappa=0.92$)			Relation ($\kappa=0.94$)	
Type	F1	n	F1	n
Abbreviation	1.00	5	1.00	5
Developer	0.93	85	0.98	78
Extension	1.00	4	1.00	4
Alternative name	1.00	2	1.00	2
Citation	0.92	40	0.98	33
Release	0.89	5	1.00	4
URL	1.00	16	0.88	16
Version	0.98	136	0.99	132
Overall	0.96	293	0.96	269
Plugin	–	–	1	11
Specification	–	–	1	5

Creation sections were not considered because the annotations process differed from the other sets as described in Section 4.1. The IRR was assessed by FScore, as suggested by Hripcsak and Rothschild [10], as agreement by chance can be neglected in exact match text annotations, and Cohen’s κ as categorical errors can still be present within the annotation categories. Note that partially overlapping annotations were not considered as matching.

Entity annotation for the first step, including both software type and mention type, resulted in a $\kappa=0.82$. Detailed results for agreement are given in Table 2. Operating systems, programming environments and creation statements have perfect agreement. Agreement is also high for application and usage with 0.94 FScore and 0.95 FScore. For plugin and allusion it was lower but with 0.72 FScore and 0.84 FScore still on a reasonable level.

For the second step, annotation of additional information, such as the version or developer, annotators were provided with the same base annotation of software mentions. Objective was to label all additional information and their reference. In summary, inter-rater reliability for the additional information and relations was almost perfect ($\kappa=0.92$ and $\kappa=0.94$). Details are provided in Table 3.

The IRR for disambiguation was initially assessed on 10% of software entity annotations with 68% agreement. As described in Section 4.2, final disambiguation was obtained by merging the result of two annotators. The following reasons were found for the low reliability: (1) different websites for the same resources, (2) resource locations changing over time, (3) canonical links vs standard links, (4) old software without valid, persistent identifier, and (5) information behind pay walls.

5 SOMESCI KNOWLEDGE GRAPH - FACTS & AVAILABILITY

The SoMeSci knowledge graph consists of 399,942 triples representing 47,524 sentences from 1367 documents from 4 datasets. 2728 of those sentences contain at least one entity, resulting in 7237 labelled entities of the different types. The remaining sentences are included as negative examples. In total, 3756 software mentions of all types were found. A detailed overview of the frequencies of the different label types is provided in Table 4.

For each software, as well as for developers, citations, and licenses, the identity was annotated to provide a means for entity disambiguation, see Section 4. If available, a URL was used to further provide a means for Entity Linking tasks. Out of 637 unique software for all types, 432 unique citations, and 9 unique licenses, 22 respectively 3 and 1 could not be linked. All 10 unique operating systems and 24 unique programming environments could be linked.

The dataset is available in three formats:

- a SPARQL endpoint (<https://data.gesis.org/somesci>) with an overview and example queries of the resource to interactively explore the knowledge graph,
- an RDF graph serialized into JSON-LD format for archival to ensure long-time availability of the resource, and
- the raw files that were created by manual annotation and used to create the above graph. The annotation of entities and relations are provided in stand-off format, while the entity's identity and links are given in JSON.

The latter two are available for download under CC-BY 4.0 license from Zenodo [23]. All source code used to process the texts, the annotation and the links is publicly available at Github (https://github.com/dave-s477/SoMeSci_Code). This also includes a tutorial and code for replicating the use cases described below. Along with the SPARQL Endpoint we also provide the queries used for all analyses described in Section 6.2 as a starting point for further exploration. We encourage community feedback through Github issues as it provides a transparent way to incorporate resource updates that will consistently be integrated in the KG.

6 USE CASES AND IMPACT

As described above, SoMeSci is the first dataset of software mentions in scientific articles that classifies software into respective types, and provides additional information about the software and their identity, paving the way for various use cases, including NLP tasks concerned with scholarly software use or the analyses of software citation practices. This can either be done by directly analyzing SoMeSci, or through supervised NLP models when used as training/test data. In the following, we first illustrate the training of

information extraction models by providing baseline recognition results and then sketch analyses concerned with the completeness of software citations, facilitated through models trained on our corpus. Note that our baseline implementation serves to illustrate potential use cases for SoMeSci, rather than establishing state-of-the-art recognition performance.

6.1 Facilitated Information Extraction Tasks

As shown in Section 2, information extraction from scholarly articles is a crucial challenge requiring targeted models for specific kinds of scholarly information. In particular, extraction and disambiguation of software mentions through supervised models has been an area of increasing interest. Given the comprehensiveness of our model and data, containing marked mentions in natural text, their types and disambiguation to actual software artefacts as defined in a background knowledge base (Wikidata), SoMeSci facilitates three basic use cases: (1) Named Entity Recognition (NER), (2) Relation Extraction (RE), and (3) Entity Disambiguation (ED). In this section, we introduce baseline implementations facilitated by our data. For training, the corpus of articles was split into training, development and test set in a 60:20:20 ratio for each individual subset of files (see Section 4.1). The 677 individual PLoS sentence were not split but instead added to the train set to prevent the overestimation of the identification due to the ratio shifted towards common software names.

6.1.1 Named Entity Recognition. NER is performed for: (1) Software in general, (2) software type, (3) mention type, and (4) all additional information. The baseline is established by the Bi-LSTM-CRF model described in Schindler et al. [25] and a separate model is trained for each task. Articles are tagged sentence-wise and each sentence is processed as a sequence of tokens. Tokens are transformed into feature representations by concatenating word embeddings [17] trained on PMC OA and character based feature representations calculated by a Bi-LSTM to learn from a token's orthography. The features are processed by a Bi-LSTM, which captures the surrounding context of entities. To generate a tag sequence a CRF in combination with a fully connected layer and final softmax classification is used. The CRF determines the most likely sequence of tags for a feature sequence, enforcing tagging consistencies that are unknown to lower layers of the classifier. The Bi-LSTM-CRF was implemented in PyTorch v1.4.0 [20] and the Word2Vec model with Gensim v3.8.1 [21]. All models were trained with the standard configuration [25] with early stopping based on the development set. Without further optimization, a Precision of 0.85 (0.82), Recall of 0.81 (0.77), and Fscore of 0.83 (0.79) was achieved for test and development set. An overview of the results is given in Table 5.

To test the potential generalisation of models trained on SoMeSci to previously unseen full text articles from PMC, training was performed in a 5-fold cross validation on the annotated full texts. An FScore of .791 ($\pm .039$) was achieved with 4 training runs on each fold. Given that the baseline result is within the standard deviation of the Fscore, this demonstrates the generalisability of the corpus and suggests it as training corpus for full text analysis on PMC.

6.1.2 Relation Extraction. RE is performed for all software-related entities within one sentence by using a Random Forest classifier

Table 4: Overview of the frequency of the different label types. Coreference refers to an annotated software coreference in the scope of a software deposition statement as described in Section 3.1. Note that only the most specific Mention Type is counted.

Software Type	Mention Type				Additional Information											
	Allusion	Usage	Creation	Deposition	Version	Developer	Extension	Alt. Name	Citation	Release	URL	License	Abbreviation	Specification	Plugin	
Application	193	2206	197	102	1221	766	47	31	407	50	236	34	68	60	68	
Plugin	51	275	70	29	30	12	–	4	141	1	54	12	12	–	6	
OS	30	143	–	–	31	9	4	–	–	1	–	–	–	–	–	
PE	47	374	–	–	70	73	–	–	42	35	8	–	3	5	134	
Coreference	–	–	–	39	–	–	–	–	1	–	40	6	–	–	–	
Overall	321	2998	267	170	1352	860	51	35	591	87	338	52	83	65	208	

Table 5: Summary of NER results. Mention type is only considered correct if the most specific sub-type is recognized.

		Application					Software type					Mention Type										Additional Info
		Plugin	OS	PE	Co-reference		Allusion	Usage	Creation	Deposition		Abbreviation	Developer	Extension	Alt. Name	License	Citation	Release	URL	Version		
Test	Precision	.79	.64	1.0	.98	.00	.80	.68	.77	.87	.76	.77	.91	.88	.60	.25	.91	.92	.88	.91	.92	.89
	Recall	.84	.32	.70	1.0	.00	.77	.18	.84	.51	.57	.72	.59	.87	.60	.25	.71	.82	.78	.98	.92	.87
	FScore	.81	.43	.82	.99	.00	.78	.29	.80	.64	.65	.74	.71	.88	.60	.25	.80	.87	.82	.95	.92	.88
	Support	415	78	30	63	4	590	71	438	53	28	590	17	110	5	4	14	120	9	53	190	522
Devel	Precision	.73	.65	1.0	.95	.75	.74	.68	.78	.97	.79	.79	.89	.91	1.0	.25	1.0	.93	.67	.93	.94	.89
	Recall	.80	.27	1.0	.95	.33	.70	.28	.80	.50	.53	.67	.47	.93	.60	.17	.80	.78	.86	.94	.98	.85
	FScore	.76	.39	1.0	.95	.46	.72	.40	.79	.66	.63	.71	.62	.92	.75	.20	.89	.85	.75	.94	.96	.87
	Support	303	113	19	55	9	499	67	336	60	36	499	17	87	5	6	10	89	7	70	139	430

with manually engineered input features. For each pair of entities and its context, the following features were considered: (1) entity order, (2) entity types, (3) entity length in tokens and characters, (4) entity distance in tokens and characters, (5) sub-strings relation between entities, and (6) acronym relation between entities. Acronyms were generated by string normalization, exclusion of stop words, and removal of trailing numbers. RE is evaluated by Precision, Recall and FScore and summarized in Table 6. Overall, relations were extracted with a performance of .88 FScore.

6.1.3 Disambiguation. Software entities are disambiguated by hierarchical clustering. The method was advanced from the approach described in Schindler et al. [25] by considering the available additional information. The following features were computed for all pairs of entity mentions: (1) Levenshtein distance and sub-string relation between entity strings, associated URLs, and associated Developers, (2) known DBpedia alternative software names as a form of distant supervision, (3) Levenshtein distance on normalized strings (casing, removal of stop words and trailing numbers), and (4) exact match comparison of automatically generated acronyms from normalized strings. The individual linking steps based on

the feature matrices were also manually engineered and their details can be found in the published implementation, see Section 5. Entity disambiguation was evaluated based on all entity pairs by comparing predicted links between pairs against the gold standard.

Considering all possible entity tuples, we achieve Precision 0.99, Recall 0.96 and FScore 0.97. As this includes all mentions where the string is an exact match, the results were also calculated excluding exact matches: Precision 0.96, Recall 0.91, FScore 0.93. However, with exact string matches a false positive case was found for the software mention “VBA”, once referring to Visual Basic for Applications (<https://www.wikidata.org/wiki/Q667566>) and three times to the Variational Bayesian Analysis toolbox (<https://mbb-team.github.io/VBA-toolbox/>).

6.2 Software in Scholarly Articles

The recently published software citation principles [12] state that a software citation should be accompanied by the version, the developer and a URL. By employing SoMeSci as a training set for information extraction, a large scale knowledge graph could be constructed to gain an understanding of software citation in science. By querying the SoMeSci knowledge graph directly, we

Table 6: Summary of RE results for relating an entity to its base entity.

		Abbreviation	Developer	Extension	AlternativeName	License	Citation	Release	URL	Version	Plugin	Specification	Relations
Test	Precision	1.0	.94	1.0	1.0	.88	.87	.67	.92	.97	.76	1.0	.92
	Recall	.94	.93	.80	1.0	.50	.84	.44	.87	.93	.56	.12	.85
	FScore	.97	.94	.89	1.0	.64	.86	.53	.89	.95	.65	.22	.88
	Support	17	111	5	4	14	121	9	53	190	39	8	571
Devel	Precision	1.0	.91	1.0	.83	1.0	.87	.75	.74	.96	.62	.75	.87
	Recall	1.0	.99	1.0	.83	.40	.79	.86	.87	.96	.57	.50	.87
	FScore	1.0	.95	1.0	.83	.57	.83	.80	.80	.96	.60	.60	.87
	Support	17	87	5	6	10	90	7	70	139	35	6	472

found that out of 3123 software mentions (application or plugin) in 1090 documents, only 1227 (39%) are accompanied by a version, 733 (23%) by a developer, 132 (4%) by a URL, and 513 (16%) by a formal citation. When considering version and developer at the same time, the number decreases to 520 (17%) software mentions in 427 documents. When looking at software depositions, out of 170 statements in 105 articles, 164 (96%) are accompanied by a URL, 5 (3%) by a version, and 49 (29%) by a license. By exploiting the linked nature, e.g. in combination with the MAKG [6] or by applying supervised extraction pipelines trained on our corpus, further exploration of changes over time can be done.

7 LIMITATIONS

SoMeSci was created through a careful manual annotation process of OA articles from PLoS and PubMed Central. While the IRR reflects very good agreement ($\forall \kappa \approx 0.8$, $\forall F1 > 0.72$), this does not imply that all entities of interest were discovered. This is also true for disambiguation. A challenging aspect for the annotation was distinguishing between applications and plugins. Similarly, the distinction between mentions of algorithms and software was hard, as authors tend to name software based on the implemented algorithm. We found that external knowledge was required, usage of which is also suggested for automatic software recognition.

Through its selection, see Section 4.1, SoMeSci is biased towards methods sections. However, an analysis of PubMed fulltexts showed that 86% of software mentions do appear in methods sections. Additionally, we were able to show that the corpus generalizes to unseen full text articles, see Section 6.1.1.

The frequencies for some types of entities are still low despite enrichment, for instance, with software creation statements. We believe that this reflects the general rareness of those entities within articles of the represented domains. Furthermore, the distribution of software and mention types might be a direct result from the selection of articles. The Softcite [3] dataset shows that software is potentially used less or cited less in economics.

Finally, we observed few cases (<10) where software and provided additional information stood in obvious conflict, e.g. the provided developer does not reflect the actual developer. We decided

to still annotate the information to represent the author's intent to provide additional information for the software.

8 CONCLUSION

SoMeSci is a gold standard knowledge graph that covers software type, mention type and additional information, as well as their textual relations in scholarly articles. It covers many concepts regarding software citations that have previously only been investigated individually or have not yet been explored, e.g. different mention types and linking of entity names. SoMeSci was created by manually annotating 1367 PMC OA articles covering a broad range of publications in life sciences and related disciplines. Identified entities were linked to unique identifiers. The high quality of all manual annotation processes is underlined by very good agreement regarding textual annotations.

We believe that SoMeSci is a valuable resource for the community as only few data for software mentions in scholarly texts is publicly available. We sketched different use cases by performing analyses about software mentions contained in SoMeSci and their completeness. We assume that the corpus will find most application as a gold standard for supervised machine learning and will contribute to establish an overview of the software infrastructure underlying the scientific progress. To this end, baseline results for automatic information extraction were established. While baseline results are meant to demonstrate how the data may contribute to high-quality information extraction models, more sophisticated models will be able to improve the performance and enable further large-scale analysis of software usage in science.

Acknowledgements. We thank Tazin Hossain and Hafiz Ghazi Sultan for their help with the annotation. This work was financially supported by the Deutsche Forschungsgemeinschaft (DFG, German Research Foundation) as part of the projects SFB 1270/2 (grant: 299150580) and ScienceLinker (grant: 404417453).

REFERENCES

- [1] Alice Allen, Peter J. Teuben, and P. Wesley Ryan. 2018. Schroedinger's Code: A Preliminary Study on Research Source Code Availability and Link Persistence in

- Astrophysics. *The Astrophysical Journal Supplement Series* 236, 1 (May 2018), 10. <https://doi.org/10.3847/1538-4365/aab764>
- [2] Tim Berners-Lee. 2010. Is your linked open data 5 star? <http://www.w3.org/DesignIssues/LinkedData#fivestar>
- [3] Caifan Du, Johanna Cohoon, Patrice Lopez, and James Howison. 2021. Soft-cite dataset: A dataset of software mentions in biomedical and economic research publications. *Journal of the Association for Information Science and Technology* 72, 7 (2021), 870–884. <https://doi.org/10.1002/asi.24454> arXiv:<https://arxiv.org/abs/2012.01511>
- [4] Geraint Duck, Goran Nenadic, Andy Brass, David L Robertson, and Robert Stevens. 2013. bioNerDS: exploring bioinformatics' database and software use through literature mining. *BMC bioinformatics* 14, 1 (2013), 194. <https://doi.org/10.1186/1471-2105-14-194>
- [5] Geraint Duck, Goran Nenadic, Michele Filannino, Andy Brass, David L Robertson, and Robert Stevens. 2016. A survey of bioinformatics database and software usage through mining the literature. *PLoS one* 11, 6 (2016), e0157989. <https://doi.org/10.1371/journal.pone.0157989>
- [6] Michael Färber. 2019. The Microsoft Academic Knowledge Graph: A Linked Data Source with 8 Billion Triples of Scholarly Data. In *The Semantic Web – ISWC 2019*. Springer International Publishing, 113–129. https://doi.org/10.1007/978-3-030-30796-7_8
- [7] Daniel Garijo, Maximiliano Osorio, Deborah Khider, Varun Ratnakar, and Yolanda Gil. 2019. OKG-Soft: An Open Knowledge Graph with Machine Readable Scientific Software Metadata. In *2019 15th International Conference on eScience (eScience)*. IEEE, 349–358. <https://doi.org/10.1109/escience.2019.00046>
- [8] Sebastian Hellmann, Jens Lehmann, Sören Auer, and Martin Brümmer. 2013. Integrating NLP Using Linked Data. In *Advanced Information Systems Engineering*. Springer Berlin Heidelberg, 98–113. https://doi.org/10.1007/978-3-642-41338-4_7
- [9] James Howison and Julia Bullard. 2016. Software in the scientific literature: Problems with seeing, finding, and using software mentioned in the biology literature. *Journal of the Association for Information Science and Technology* 67, 9 (2016), 2137–2155. <https://doi.org/10.1002/asi.23538>
- [10] George Hripcsak and Adam S Rothschild. 2005. Agreement, the f-measure, and reliability in information retrieval. *Journal of the American medical informatics association* 12, 3 (2005), 296–298. <https://doi.org/10.1197/jamia.M1733>
- [11] Mohamad Yaser Jaradeh, Allard Oelen, Kheir Eddine Farfar, Manuel Prinz, Jennifer D'Souza, Gábor Kismihók, Markus Stocker, and Sören Auer. 2019. Open Research Knowledge Graph: Next Generation Infrastructure for Semantic Scholarly Knowledge. In *Proceedings of the 10th International Conference on Knowledge Capture (Marina Del Rey, CA, USA) (K-CAP '19)*. Association for Computing Machinery, New York, NY, USA, 243–246. <https://doi.org/10.1145/3360901.3364435>
- [12] Daniel Katz, Neil Chue Hong, Tim Clark, August Muench, Shelley Stall, Daina Bouquin, Matthew Cannon, et al. 2021. Recognizing the value of software: a software citation guide. *F1000Research* 9 (Jan. 2021), 1257. <https://doi.org/10.12688/f1000research.26932.2>
- [13] Frank Krüger and David Schindler. 2020. A Literature Review on Methods for the Extraction of Usage Statements of Software and Data. *Computing in Science & Engineering* 22, 1 (Jan. 2020), 26–38. <https://doi.org/10.1109/mcse.2019.2943847>
- [14] Kai Li, Erjia Yan, and Yuanyuan Feng. 2017. How is R cited in research outputs? Structure, impacts, and citation standard. *Journal of Informetrics* 11, 4 (2017), 989–1002. <https://doi.org/10.1016/j.joi.2017.08.003>
- [15] James Malone, Andy Brown, Allyson L Lister, Jon Ison, Duncan Hull, Helen Parkinson, and Robert Stevens. 2014. The Software Ontology (SWO): a resource for reproducibility in biomedical data analysis, curation and digital preservation. *Journal of Biomedical Semantics* 5, 1 (2014), 25. <https://doi.org/10.1186/2041-1480-5-25>
- [16] Paolo Manghi, Alessia Bardi, Claudio Atzori, Miriam Baglioni, Natalia Manola, Jochen Schirrwagen, and Pedro Principe. 2019. The OpenAIRE Research Graph Data Model. <https://doi.org/10.5281/ZENODO.2643199>
- [17] Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, and Jeffrey Dean. 2013. Distributed Representations of Words and Phrases and Their Compositionality. In *Proceedings of the 26th International Conference on Neural Information Processing Systems - Volume 2 (Lake Tahoe, Nevada) (NIPS '13)*. Curran Associates Inc., Red Hook, NY, USA, 3111–3119.
- [18] U. Nangia and D. S. Katz. 2017. Understanding Software in Research: Initial Results from Examining Nature and a Call for Collaboration. In *2017 IEEE 13th International Conference on e-Science (e-Science)*. IEEE, 486–487. <https://doi.org/10.1109/eScience.2017.78>
- [19] Xuelian Pan, Erjia Yan, Qianqian Wang, and Weina Hua. 2015. Assessing the impact of software on science: A bootstrapped learning of software entities in full-text papers. *Journal of Informetrics* 9, 4 (2015), 860–871. <https://doi.org/10.1016/j.joi.2015.07.012>
- [20] Adam Paszke, Sam Gross, Francisco Massa, Adam Lerer, James Bradbury, Gregory Chanan, Trevor Killeen, et al. 2019. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems* 32. Curran Associates, Inc., 8024–8035.
- [21] Radim Rehurek and Petr Sojka. 2010. Software Framework for Topic Modelling with Large Corpora. In *Proceedings of the LREC 2010 Workshop on New Challenges for NLP Frameworks*. ELRA, Valletta, Malta, 45–50.
- [22] Jason Ronallo. 2012. HTML5 Microdata and Schema. org. *Code4Lib Journal* 16 (2012).
- [23] David Schindler, Felix Bensmann, Stefan Dietze, and Frank Krüger. 2021. SoMeSci. <https://doi.org/10.5281/zenodo.4968738>
- [24] David Schindler, Kristina Yordanova, and Frank Krüger. 2019. An annotation scheme for references to research artefacts in scientific publications. In *2019 IEEE International Conference on Pervasive Computing and Communications Workshops (PerCom Workshops)*. IEEE, 52–57. <https://doi.org/10.1109/PERCOMW.2019.8730730>
- [25] David Schindler, Benjamin Zapilko, and Frank Krüger. 2020. Investigating Software Usage in the Social Sciences: A Knowledge Graph Approach. In *ESWC 2020: The Semantic Web*. Springer International Publishing, Cham, 271–286. https://doi.org/10.1007/978-3-030-49461-2_16
- [26] Arfon M Smith, Daniel S Katz, and Kyle E Niemeyer. 2016. Software citation principles. *PeerJ Computer Science* 2 (2016), e86. <https://doi.org/10.7717/peerj-cs.86>
- [27] Pontus Stenetorp, Sampo Pyysalo, Goran Topić, Tomoko Ohta, Sophia Ananiadou, and Jun'ichi Tsujii. 2012. BRAT: a Web-based Tool for NLP-Assisted Text Annotation. In *Proceedings of the Demonstrations at the 13th Conference of the European Chapter of the Association for Computational Linguistics*. Association for Computational Linguistics, Avignon, France, 102–107. <https://www.aclweb.org/anthology/E12-2021>